240410206 Nkadimeng M

This semester, our group moved from using traditional Java EE to a modern system using React and Spring Boot. My main job was to build the Staff Dashboard, especially the Live Queue Operations section of the QueueMasters system.

This part of the system allows staff at the Witbank auto-care branch to manage cars in the queue in real time. Working on this helped me understand how modern web systems handle live updates, user interaction, and dynamic data.

From Old System to Modern System

In Java EE projects, pages usually reload every time a user does something. The server processes the request and sends back a new page.

In this project, we used React, which creates a Single Page Application (SPA). This means:

- **The page does not reload.**
- **Data updates automatically.**
- **The browser communicates with the backend using APIs.**

This changed how I think about development. Instead of refreshing pages, I now focus on updating the application's state so the UI changes instantly.

Building the Live Queue Table

The main feature of my page is a table that shows:

- **Car plate number**
- **Car model**
- **Customer contact details**
- **Current service status (like "IN QUEUE" or "WASHING")**
- **Action buttons (Next, Edit, Release)**

I used React's .map() function to display data from the backend. I also used the useState hook to store and update the queue data.

At first, I learned that directly changing state does not update the screen. I had to use the correct method (setState) to make React re-render the UI.

Implementing the Action Buttons

1. Next Button

This moves a car forward in the queue (for example, from "IN QUEUE" to "WASHING").

To do this, I:

- **Sent an update request to the backend**
- **Updated the database**
- **Updated the frontend state**

This helped me understand how to handle asynchronous API requests using Axios.

---

2. Edit Button

This allows staff to update car or customer details.

I learned how to:

- **Pre-fill form data**
- **Pass selected data into an edit form**
- **Update records dynamically**

This improved my understanding of form handling and data binding.

---

3. Release Button

This removes a car from the queue after service is complete.

I had to:

- **Add confirmation before deleting**
- **Make sure the UI updates immediately**

This taught me how to prevent mistakes and keep the system consistent.

Real-Time Updates

To keep the queue updated, I used a polling system with the useEffect hook. The system fetches new data regularly.

At first, I had problems with multiple intervals running at the same time, which caused performance issues. I fixed this by cleaning up the interval properly.

This taught me about:

- **React component lifecycle**
- **Memory leaks**
- **Proper resource management**

Improving User Experience

Since staff use this system in a busy environment, it must be clear and fast.

I added:

- **Color-coded status labels**
- **Clear action buttons**
- **Clean table layout**

I learned that good design makes systems easier and faster to use.

Working with the Backend

I connected the dashboard to Spring Boot REST APIs. The frontend and backend were separate, so we had to agree on the JSON data structure.

This improved:

- **My teamwork skills**
- **My understanding of API contracts**
- **My knowledge of modern decoupled architecture**

Challenges and Lessons

I faced challenges like:

- **Users clicking buttons too quickly**
- **Backend failures affecting the UI**
- **Debugging asynchronous code**

From this, I learned to:

- **Disable buttons when needed**
- **Use try/catch for error handling**
- **Think from the user's perspective**

Comparing to Java EE

Compared to Java EE, this modern system:

- **Is more interactive**
- **Updates without refreshing pages**
- **Allows frontend and backend to work independently**

It provides:

- **Faster user experience**
- **Better scalability**
- **Easier maintenance**

Conclusion

Developing the Staff Dashboard was one of the most important experiences of this semester. I learned about:

- **State management**
- **API communication**
- **Real-time systems**
- **Modern frontend development**

Most importantly, I changed how I think about building applications. I now understand that the frontend plays a big role in controlling workflows and improving user experience.

This project has prepared me for real-world software development, where speed, usability, and system integration are very important.